

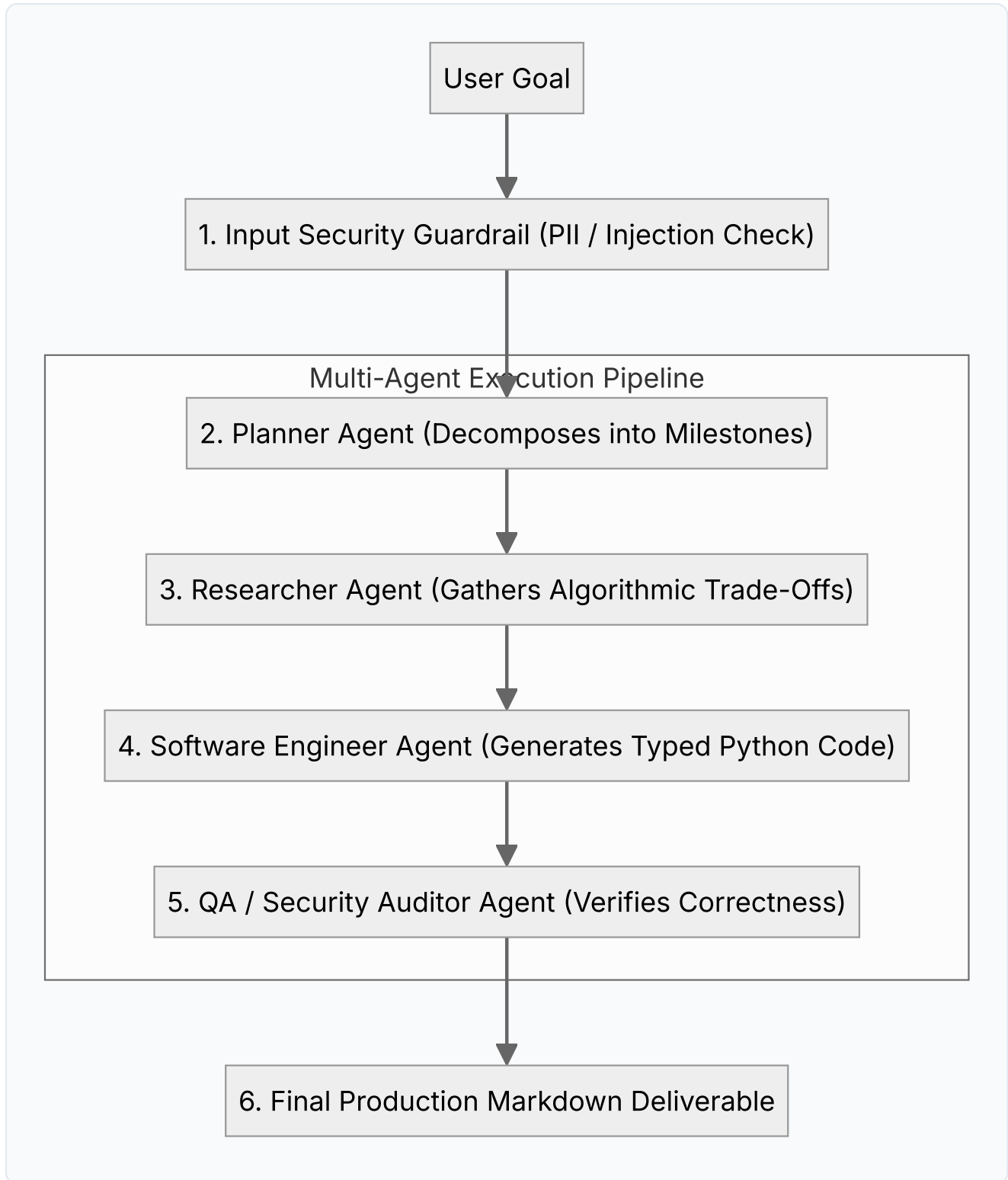
Agentic AI Chapter 8: Capstone Project: Autonomous Research & Code Assistant

Core Learning Objective: Bring everything together in an intuitive, non-overwhelming end-to-end capstone project. Build an autonomous multi-agent pipeline from scratch in pure Python with zero external paid APIs or framework churn.

1. Project Overview & Architectural Flow

We will build an **Autonomous Technical Research & Code Generation Agent**. Given a user's high-level request (e.g., *"Build an in-memory sliding window rate limiter in Python"*), the system autonomously:

1. **Plans:** Decomposes the user prompt into structured research and coding milestones.
2. **Researches:** Queries a knowledge base of algorithm trade-offs.
3. **Codes:** Writes clean, typed Python implementation code.
4. **Audits:** Validates the generated code against security vulnerabilities and correctness checks.
5. **Streams Output:** Delivers a complete, audited engineering deliverable.



2. Runnable Capstone Skeleton (Pure Python, Simulated LLM Calls)



How to Turn the Skeleton into a Real Agent

The capstone is a skeleton with simulated model calls. To make it real: (1) implement `get_llm()` for your provider (see `examples/llm.py`, which has Anthropic and OpenAI adapters behind `LLM_PROVIDER=anthropic|openai` (set `LLM_MODEL` to a current model id; both are unit-tested with stub clients)), (2) keep a `FakeLLM` script for each scenario so tests stay offline and deterministic, (3) add the evaluation loop from `examples/ex05_eval_harness.py` (repeat each task 5 to 10 times and report `pass^k` with a confidence interval), and (4) gate side effects behind the approval pattern from `examples/ex02_langgraph_hitl.py`.

4. Runnable Model: Acceptance Tests for the Capstone

A capstone is only finished when it can be graded without opinion. These checks are deterministic, run offline, and are the kind of automated gate a review would apply to your agent's output.

Rubric the grader (human or script) applies

Area	Passing evidence	Automated check
Grounding	Every factual claim carries a citation that exists	<code>check_report</code> above
Tool discipline	No calls outside the allowlist; arguments validated; step and cost budgets enforced	Replay the trace and assert limits
Recovery	A failed tool call leads to a retry, an alternative or a clear partial answer, never a crash	Inject a failing tool in a test and assert the final status
Safety	Untrusted web content cannot trigger a write or send action	Feed a hostile page and assert no privileged tool was called
Reproducibility	The same inputs and recorded model outputs give the same report	Run the pipeline twice against a recording
Cost	Cost per task is reported and under the stated cap	Sum token usage from the trace

Suggested milestones

1. **Day 1, the loop:** a ReAct loop with two tools (search stub, calculator), a step budget and a trace log, tested with a scripted model.

2. **Day 2, grounding:** retrieval over a small fixed corpus, citations in the output, `check_report` as a test.
3. **Day 3, safety and recovery:** tool allowlist and argument validation, a failing-tool test, a hostile-page test.
4. **Day 4, evaluation:** 20 golden tasks, a pass-rate gate with a confidence bound, and a cost report.
5. **Day 5, polish:** README with architecture, limits, a demo transcript and the evaluation results.

The deliverable that distinguishes strong submissions is not a clever prompt; it is the **evidence**: tests, a trace, an evaluation table and an honest list of known failures.

Further Reading

- [Anthropic: building effective agents](#)
 - [LangGraph documentation](#)
 - [SWE-bench](#)
-

Check Yourself

Answer in your head or on paper first, then open each answer.

- ▶ 1. What makes a capstone agent testable offline?
- ▶ 2. Which three checks gate a side-effecting action?
- ▶ 3. What should the capstone log?
- ▶ 4. How do you turn a simulation into a real agent?